EXP:NO:06 HAMMING CODE

DATE : 11/08/26

AIM:

Write a program to implement error detection and correction using HAMMING Code Concept.

Create sender program

Sender.py

```
def find_r(k):
    r = 0
    while 2**r < k + r + 1:
        r += 1
    return r
text = input("Enter text: ")
binary = ""
for ch in text:
    binary += format(ord(ch), "08b")
print("Binary:", binary)
k = len(binary)
r = find_r(k)
print("Data bits:", k)
print("Redundancy bits:", r)
n = k + r
code = [0] * (n+1)
j = 0
```

```
for i in range (1,n+1):
    if i & (i-1) != 0:
        code[i] = int(binary[j])
        j += 1
p = 1
while p <= n:
    parity = 0
    for i in range (p, n+1):
        if i & p:
            parity ^= code[i]
    code[p] = parity
    p = p*2
hamming_code = ""
for i in range (1,n+1):
    hamming_code += str(code[i])
print("Hamming Code :", hamming_code)
with open("channel.txt", "w") as f:
    f.write(str(k) + "\n")
    f.write(hamming_code)
print("Data saved in channel.txt")
```

Receiver.py

```
def binary_to_text(binary):
    text = ""
    for i in range (0, len(binary), 8):
        byte = binary[i:i+8]
        if len(byte) == 8:
            text += chr(int(byte,2))
    return text
with open("channel.txt","r") as f:
    k = int(f.readline())
    data = f.readline().strip()
print("Received Hamming Code:",data)
n = len(data)
code = [0] + [int(x) for x in data]
corrupted_binary = ""
for i in range (1,n+1):
    if i & (i-1) != 0:
        corrupted_binary += str(code[i])
corrupted_binary = corrupted_binary[:k]
corrupted_text = binary_to_text(corrupted_binary)
print("Corrupted Text:", corrupted_text)
error = 0
p = 1
while p <= n:
    parity = 0
```

```
    for i in range (p, n+1):
        if [i & p :
            parity ^= code[i]

    if parity:
        error += p
    p = p * 2

if error == 0:
    print ("No error detected")
else :
    print (" Error detected at position :", error)
    code[error] = 1 - code[error]
    print (" Error corrected.")

Corrected_binary = ""
for i in range (1, n+1):
    if i & (i-1) != 0 :
        Corrected_binary += str (code[i])

Corrected_binary = corrected_binary [: k]

Corrected_text = binary_to_text (Corrected_binary)

print ("Corrected Text :", Corrected_text)
```

OUTPUT : sender.py

```
Enter text : internet
Binary : 01101001011011100111010001100101011100100110110011100101011110100
Data bits : 64
Redundancy bits : 7
Hamming code : 00001100100101111011100111010001 0 100 1010
1110010011011100110010100111010 0
Data saved in channel.txt
```

receiver.py

```
Received Hamming code : 00001110100101110111001110100010
100 1010 1110010011011100 1100101001110100
Corrupted text : ynternet
Error detected at position : 7
Error corrected.
Corrected Text : internet
```

7 bits for parity
corrected bits exclusively

RESULT:

Thus, error detection and correction is done using hamming code.